西北工业大学实验报告

课程名称: 高级语言程序设计实验

实验课次周次: 第一次（第七周）

实验地点: 教学西楼 B2-201

学生姓名: 张洛宾

学　　号: 2024302696

实验时间: 2026-04-18

提交时间: 2026-04-23

成绩评定：（满分为 100 分） 指导教师评语： 指导教师签名：

目 录

1 JAVA 实验一 熟悉 JAVA 程序的开发 2

1.1 实验内容（要求、算法、步骤和方法） 2

1.2 JAVA 源代码文件说明 2

1.3 JAVA 实验结果 2

2 JAVA 实验二 面向对象的编程技术 3

2.1 实验内容（要求、算法、步骤和方法） 3

2.2 JAVA 源代码文件说明 4

2.3 JAVA 实验结果 4

3 JAVA 实验三 类的方法 8

3.1 实验内容（要求、算法、步骤和方法） 8

3.2 JAVA 源代码文件说明 9

3.3 JAVA 实验结果 9

4 JAVA 实验四 类的重用 15

4.1 实验内容（要求、算法、步骤和方法） 15

4.2 JAVA 源代码文件说明 16

4.3 JAVA 实验结果 16

5 JAVA 实验五 包、接口、类库 21

5.1 实验内容（要求、算法、步骤和方法） 21

5.2 JAVA 源代码文件说明 22

5.3 JAVA 实验结果 22

6 实验总结 29

6.1 实验中遇到的问题以及解决方法 29

6.2 实验体会 30

1 Java 实验一 熟悉 Java 程序的开发

1.1 实验内容（要求、算法、步骤和方法）

具体代码请参见附件中电子版的 java 源程序。源程序的简要说明如下：

编写并运行第一个 Java Application 程序。其它请参见实验指导书。

1.2 Java 源代码文件说明

MyClass.java —— 第一个 Java 程序。

1.3 Java 实验结果

（包括重点代码说明，思路，运行结果，以及实验要求中需要书面回答的内容）

MyClass.java 源代码如下：

```
import java.io.*;
public class MyClass
{
    private int val1,val2;
    public void myFun(int x,int y)
    {
        val1=x;
        val2=y;
        System.out.println("the sum is "+(val1+val2));
    }

    public static void main(String arg[])
    {
        MyClass MyObj=new MyClass();
        MyObj.myFun(3,4);
    }
}
```

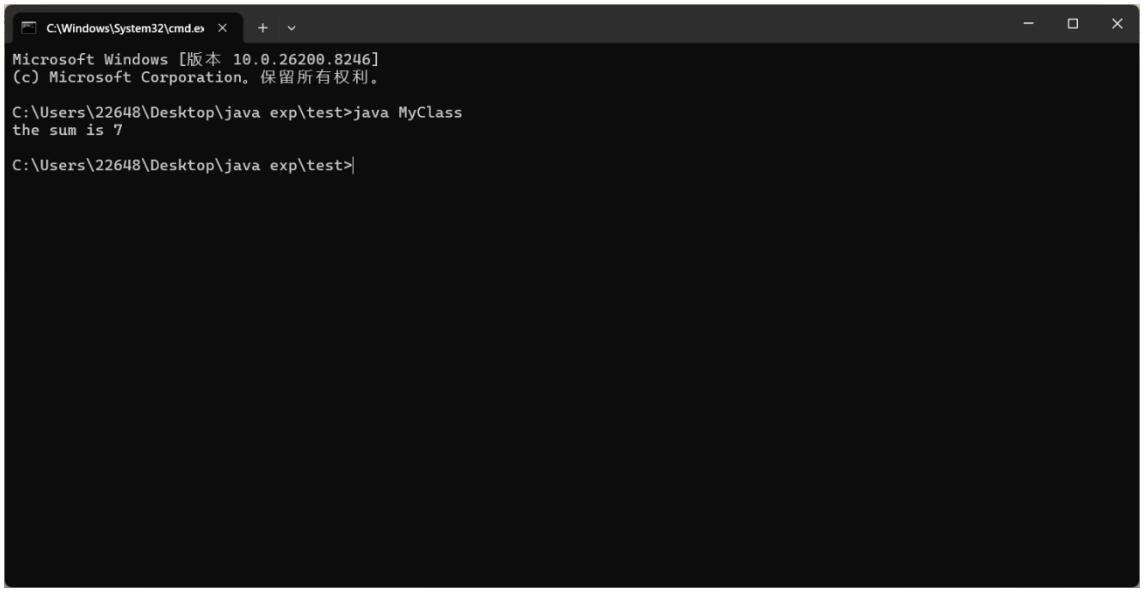
代码解析：

首先创建两个私有变量，然后创建一个方法为 myFun，输出两者的和．在主函数中创建方法实例，并且调用上述方法得到输出。

首先利用文本编辑器进行编辑，这里使用的文本编辑器是记事本，然后将

文本文件命名为MyClass.java,最后再在当前文件夹下打开cmd终端进行编译和运行。

运行结果如下：



至此实验一结束。

2 Java 实验二 面向对象的编程技术

2.1 实验内容（要求、算法、步骤和方法）

定义一个类-圆，并编一个主类测试它。

练习内容：扩展圆的定义，为其增加可以求圆面积的方法，并在主类中输出一个实例化的圆的面积。

思考问题：运行扩展后的程序，思考如下的问题。

（1） 是否可以将类-圆的定义和主类的源代码放在两个文件中。如果可以的话，两个文件的命名有何要求，上机测试后，给出结论。

（2） 修改程序，使圆的属性 pi 定义为最终变量，其值为 3.14159，看会出现什么样的结果。如果程序出错，请调整代码以适合属性 pi 为最终变量的要求。

（3） 为程序添加构造方法代码，调用该构造方法，可以完成圆的半径的初始化。

（4） 修改主类代码，测试构造方法的使用。

上机作业：

(1)编写 Book.java，定义一个类 Book，具有以下属性和方法：

属性：书名(Title)；出版日期(Pdate)；字数(Words)。

方法：计算单价 price()：单价=字数/1000*35*日期系数

上半年的日期系数=1.2；下半年的日期系数=1.18

（2）编写主类 TestClass，在主类中实例化类 Book，并输出实例化对象的属性。运行该程序，分析运行的结果，你觉得你学到了什么？

其它请参见实验指导书。

2.2 Java 源代码文件说明

具体代码请参见附件中电子版的 java 源程序。源程序的简要说明如下：

TestCCircle.java —— 主类代码,用于测试圆类,包含一个 Circile 类,有计算面积和返回半径的方法.

Book.java —— 定义书类,属性为书名(Title)；出版日期(Pdate)；字数(Words),方法为计算单价 price().

TestClass.java —— 主类代码,用于测试 Book 类

2.3 Java 实验结果

(包括重点代码说明，思路，运行结果，以及实验要求中需要书面回答的内容)

TestCCircle.java 源代码如下：

```
class CCircle   {
    double pi;
    double radius;
    double getRadius(){
        return radius;
    }
    void setCircle(double r, double p){
        pi=p;
        radius=r;
    }
}

public class TestCCircle{
    public static void main(String args[])   {
        CCircle cir1=new CCircle();
        cir1.setCircle(2.0,3.1416);
```

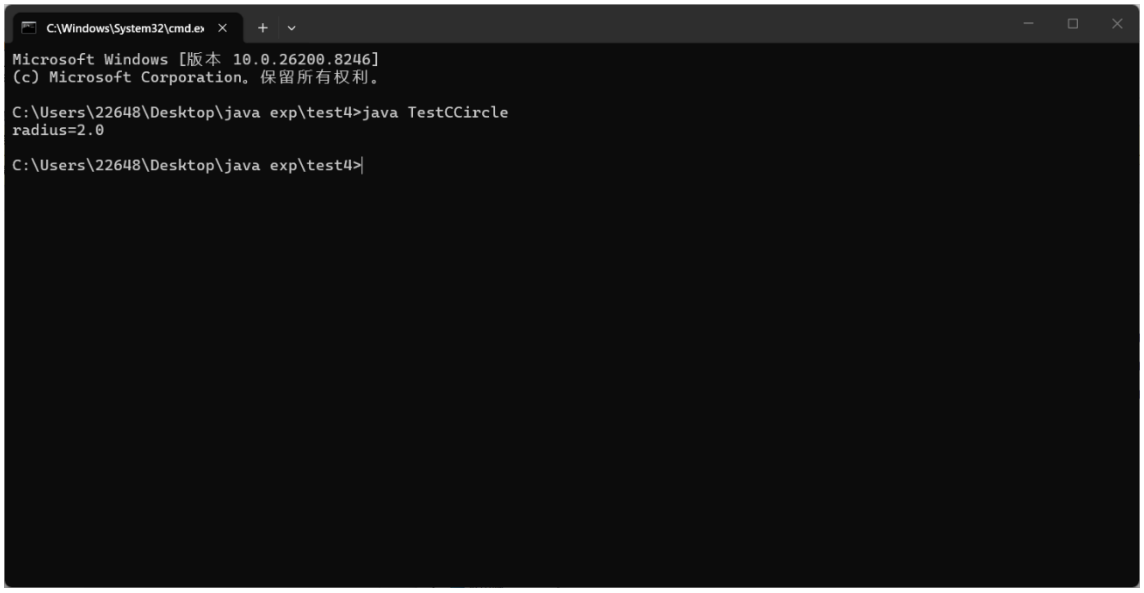
```
        System.out.println("radius="+cir1.getRadius());
    }
}
```

代码定义了一个 CCircle 类,定义了属性一个常量 PI 和一个属性为半径,实现了一个用于设置类中属性的方法,以及一个圆的面积计算代码用于计算面积.

在主类当中,创建这个 CCircle 变量的实例,并且设置其中的属性,然后计算面积和半径并且输出。

首先用记事本进行编辑 ,然后将文本文件命名为 TestCCircle.java,最后再在当前文件夹下打开 cmd 终端进行编译和运行。

运行结果如下:



思考问题:

(1) 可以,把 CCircle 类单独独立出来即可,注意 CCircle 的类名与文件名称要一致,然后 TestCCircle.java 的程序中 import CCircle 即可进行测试使用。

(2) 如果将 pi 定义为 final 变量,那么在 setCircle 这个方法中修改 pi 是不可行的会报错,因此需要去除 setCircle 中对于 pi 的操作即可。

(3) 以下为修改后的代码,去除了 setCircle 方法,添加了构造函数 CCircle,在测试类当中,直接利用构造函数构造了 CCircle 类.并且完成输出。

```
class CCircle
{
    double pi;
    double radius;
    CCircle(double radius, double pi)
    {
        this.radius=radius;
        this.pi=pi;
    }
    double getRadius()
    {
        return radius;
    }
    double getSize()
    {
        return pi*radius*radius;
    }
}
```

（4）修改后的代码如下，删除了 setCircle 方法的调用，直接使用构造方法来创建类的实例。

```
public class TestCCircle
{
    public static void main(String args[])
    {
        CCircle cir1=new CCircle(2.0, 3.1416);
        System.out.println("radius="+cir1.getRadius());
        System.out.println("Size="+cir1.getSize());
    }
}
```

上机作业：

代码如下：

```
public class Book
{
    public String title;
    public String pdate;
    public int words;

    public Book(String title, String pdate, int words) {
        this.title = title;
```

```
        this.pdate = pdate;
        this.words = words;
    }

    public double price()
    {
        String[] dateParts = pdate.split("-");
        int year = Integer.parseInt(dateParts[0]);
        int month = Integer.parseInt(dateParts[1]);
        double coefficient = (month <= 6) ? 1.2 : 1.18;
        return (words / 1000.0) * 35 * coefficient;
    }

}


public class TestClass
{
    public static void main(String[] args) {
        Book book1 = new Book("Java", "2023-05-15", 80000);
        System.out.println(book1.title +" "+ book1.price());

        Book book2 = new Book("Python", "2023-07-20", 50000);
        System.out.println(book1.title +" "+ book2.price());
    }
}
```

运行结果如下：

```
"C:\Program Files\Eclipse Adoptium\jdk-21.0.7.6-hotspot\bin\java.exe" "-javaagent:E:\IntelliJ IDEA 2025.3.4\lib\idea_rt.jar=51548" -D
Java 3360.0
Java 2065.0

进程已结束，退出代码为 0
```

分析完运行的结果之后，我了解了构造方法的重要性，能够在创建类的实例的过程中直接完成实例的初始化，而且不需要另外书写设置属性的方法. 同时我发现在同一个软件包之下，不同文件之间的类是不需要 import 语句来引入的. 且在编译的时候会一同进行编译。

至此实验二结束。

3 Java 实验三 类的方法

3.1 实验内容（要求、算法、步骤和方法）

比较两个数的大小并按升序输出。

编写程序，输出 1 到 1000 之间，所有可以被 3 整除又可以被 7 整除的数。

练习内容：创建一个具有 5 个值的数组，并找出最大值和最小值。

思考问题：

（1） 将上面的数组进行排序，数组的第一个元素为最小值，最后一个元素为最大值。

（2） 能根据给定的数组值，找出该数组值在数组中的下标。

上机作业：编写一个换算 GPA 的 Application 程序，对于学生学习的每门课程，都输入两个整数：考试成绩和学分，考试成绩按如下公式换算：

85~100：4

75~84： 3

60~74： 2

45~59： 1

44 以下：0

GPA 等于换算后每门课的成绩的学分加权平均值（Σ（成绩×学分）/Σ学分）。

学生信息参考下表

姓名	课程名	学分	成绩
张一	数学	4	71.5
张一	英语	3.5	80.4
张一	数据结构	3	95.5
李二	数学	4	78.5
李二	数据结构	3	54.5
李二	数据库	3	60.5
赵三	数学	4	88.5
赵三	英语	3.5	92.5
赵三	数据结构	3	71.5

其它请参见实验指导书。

3.2 Java 源代码文件说明

具体代码请参见附件中电子版的 java 源程序。源程序的简要说明如下：

Sort.java —— 一个从实验指导书上得到的简单的比较大小的程序。

NumTest.java —— 一个从实验指导书上得到的输出 1-1000 当中,所有可以被 3 和 7 同时整除的数。

ArrSort.java —— 练习程序,用于在五个值的数组中,找出最大最小值。

GPA.java —— 上机作业,使用多个类和集合知识实现的 GPA 换算程序。

3.3 Java 实验结果

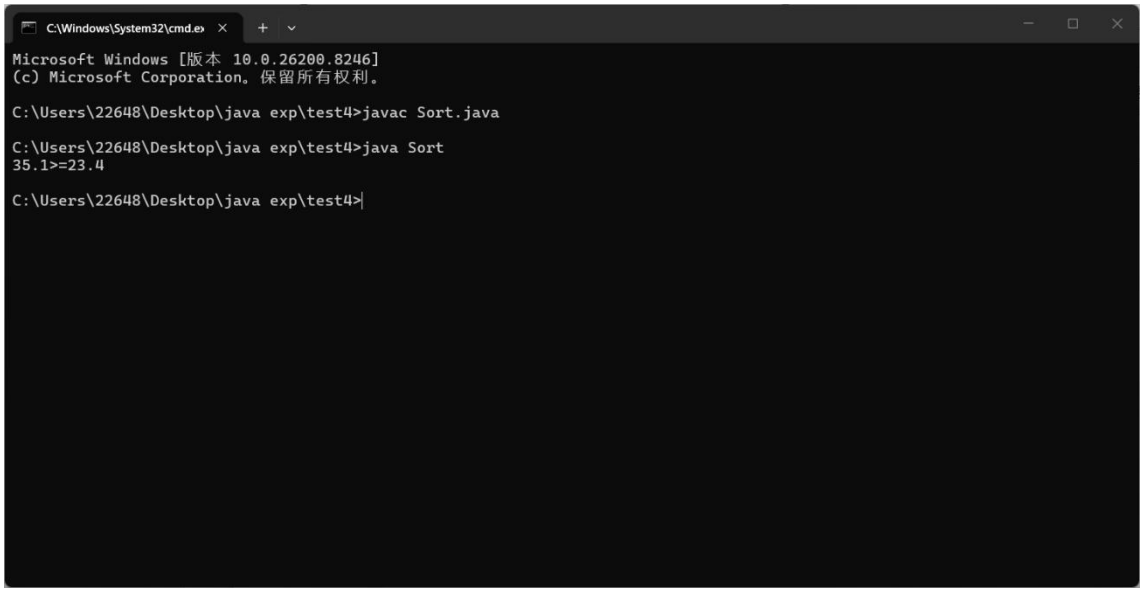
(包括重点代码说明，思路，运行结果，以及实验要求中需要书面回答的内容)

Sort.java 源代码如下：

```
public class Sort
{
    public static void main (String args[])
    {
        double d1=23.4;
        double d2=35.1;
        if (d2>=d1)
            System.out.println(d2+">="+d1);
        else
            System.out.println(d1+">="+d2);
    }
}
```

用记事本进行编辑,然后将文本文件命名为 Sort.java,最后再在当前文件夹下打开 cmd 终端进行编译和运行。

运行结果如下：

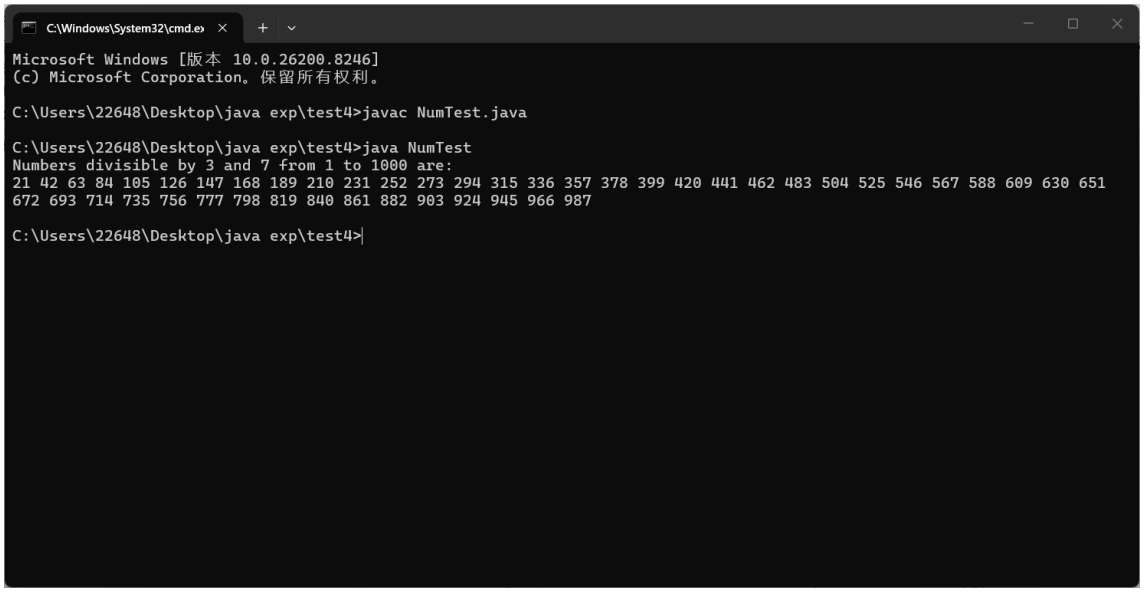


NumTest.java 源代码如下：

```
public class NumTest{
    public static void main (String args[]) {
        int n,num,num1;
        System.out.println("Numbers divisible by 3 and 7 from 1 to
1000 are:");
        for (n=1;n<=1000;n++) {
            num =n%3;
            num1=n%7;
            if (num==0) {
                if (num1==0)
                    System.out.print(n+" ");
            }
        }
        System.out.println(" ");
    }
}
```

代码用于输出 1-1000 的能被 3 和 7 整除的数.使用 for 循环和 if 来判断数字。用记事本进行编辑 ,然后将文本文件命名为 Sort.java,最后再在当前文件夹下打开 cmd 终端进行编译和运行。

运行结果如下：



练习内容的代码如下：

```
public class ArrSort
{
    public static void main(String[] args)
    {
        int[] arr =new int[5];
        int i;
        arr[0]=10;
        arr[1]=20;
        arr[2]=-9;
        arr[3]=8;
        arr[4]=98;
        int min=0,max=0;
        for(i=0;i<5;i++)
        {
            if(max<arr[i])
                max=arr[i];
            if(min>arr[i])
                min=arr[i];
        }
        System.out.println("MAX:" +max);
        System.out.println("MIN:"+min);
    }
}
```

运行结果如下：

```
"C:\Program Files\Eclipse Adoptium\jdk-21.0.7.6-hotspot\bin\java.exe" "-javaagent:E:\IntelliJ IDEA 2025.3.4\lib\idea_rt.jar=51453" -D
MAX:98
MIN:-9

进程已结束，退出代码为 0
```

思考问题：

（1） 代码如下：

```java
import java.util.Arrays; // 导入 Arrays 类
public class ArrSort
{
    public static void main(String[] args)
    {
        int[] arr = new int[5];
        arr[0] = 10;
        arr[1] = 20;
        arr[2] = -9;
        arr[3] = 8;
        arr[4] = 98;
        Arrays.sort(arr); // 对数组进行升序排序
        int min = arr[0];  // 最小值是排序后的第一个元素
        int max = arr[arr.length - 1]; // 最大值是排序后的最后一个元
素
        System.out.println("MAX: " + max);
        System.out.println("MIN: " + min);
    }
}
```

（2） 代码如下：

```java
import java.util.Arrays;
public class FindIndex
{
    public static void main(String[] args)
    {
        int[] arr = {10, 20, -9, 8, 98};
        Arrays.sort(arr); // 先排序（二分查找要求数组有序）

        int target = 8;
        int index = Arrays.binarySearch(arr, target);

        if (index >= 0) {
            System.out.println("值 " + target + " 的下标是: " +
index);
```

```
        } else {
            System.out.println("值 " + target + " 不在数组中");
        }
    }
}
```

上机作业代码如下：

```
import java.util.*;

class Course
{
    String name;
    double credit;
    double sc;
    public Course(String name, double credit, double sc)
    {
        this.name = name;
        this.credit = credit;
        this.sc = sc;
    }
}
class Student
{
    String name;
    List<Course> courses;
    public Student(String name)
    {
        this.name = name;
        this.courses = new ArrayList<>();
    }
    public void add(Course course)
    {
        courses.add(course);
    }
    public double calculateGPA()
    {
        double SumCredit = 0;
        double SumPoints = 0;

        for (Course course : courses) {
            double gradePoint = getGradePoint(course.sc);
```

```
            SumPoints += gradePoint * course.credit;
            SumCredit += course.credit;
        }
        double GPA = SumPoints / SumCredit;

        return GPA;
    }
    private double getGradePoint(double sc)
    {
        if (sc >= 85) return 4;
        else if (sc >= 75) return 3;
        else if (sc >= 60) return 2;
        else if (sc >= 45) return 1;
        else return 0;
    }
}
public class GPA
{
    public static void main(String[] args)
    {
        Student zhangYi = new Student("张一");
        zhangYi.add(new Course("数学", 4, 71.5));
        zhangYi.add(new Course("英语", 3.5, 80.4));
        zhangYi.add(new Course("数据结构", 3, 95.5));

        Student lier = new Student("李二");
        lier.add(new Course("数学", 4, 78.5));
        lier.add(new Course("数据结构", 3, 54.5));
        lier.add(new Course("数据库", 3, 60.5));

        Student zhaoSan = new Student("赵三");
        zhaoSan.add(new Course("数学", 4, 78.5));
        zhaoSan.add(new Course("英语", 3.5, 92.5));
        zhaoSan.add(new Course("数据结构", 3, 71.5));

        System.out.printf("张一的 GPA: %.2f\n",
zhangYi.calculateGPA());
        System.out.printf("李二的 GPA: %.2f\n", lier.calculateGPA());
        System.out.printf("赵三的 GPA: %.2f\n",
zhaoSan.calculateGPA());
```

```
    }
}
```

首先添加了两个类,Course 和 Student,Course 中的属性是课程人,学分和得分.然后 Student 类中有两个属性,一个是学生姓名,一个是 ArrayList 的集合,其中存放 Course 类的内容,实现了 add 方法,用于添加课程,实现了 caculateGPA 方法,用于计算平均 GPA,实现了 getCreditPoints 方法,用于将分数转换为相应绩点.在主类当中创建了三个人的实例,然后分别调用方法进行计算.最后输出结果.

运行结果如下：

```
"C:\Program Files\Eclipse Adoptium\jdk-21.0.7.6-hotspot\bin\java.exe"
张一的 GPA: 2.90
李二的 GPA: 2.10
赵三的 GPA: 3.05

进程已结束，退出代码为 0
```

至此实验三结束

4 Java 实验四 类的重用

4.1 实验内容（要求、算法、步骤和方法）

编写一个 Java 应用程序，判断两个字符是否相同，判断字符串的前缀，后缀是否和某个字符串相同，按字典顺序比较两个字符串的大小关系，检索字符串，创建子字符串，将数字型字符串转换为数字，将字符串放到数组中，用字符数组创建字符串。

上机作业：编写一个 Java 应用程序，除了主类外，该程序中还有 4 个类：People、ChinaPeople、AmericanPeople 和 BeijingPeople 类。要求如下：

- People 类有访问权限是 protected 的 double 型成员变量:height 和 weight，以及 public void speakHello()、public void averageHeight()和 public void averageWeight()方法
- ChinaPeople 类是 People 的子类，新增了 public void chinaGongfu()方法。要求 ChinaPeople 重写父类的 public void speakHello()、public void

averageHeight()和 public void averageWeight()方法

- AmericanPeople 类是 People 的子类，新增 public void americanBoxing()方法，要求 AmericanPeople 重写父类的 public void speakHello()、public void averageHeight()和 public void averageWeight()方法
- BeijingPeople 类是 ChinaPeople 的子类，新增 public void beijingOpera()方法，要求 BeijingPeople 重写父类的 public void speakHello()、public void averageHeight()和 public void averageWeight()方法

其它请参见实验指导书。

4.2 Java 源代码文件说明

具体代码模板请参见附件中电子版的 java 源程序。源程序的简要说明如下：

StringExample.java —— 实验指导书中的程序框架补充后的结果，用于对字符串进行操作

Example.java —— 实验指导书中的上机作业，主要用于练习类的继承与重写方法.

4.3 Java 实验结果

（包括重点代码说明，思路，运行结果，以及实验要求中需要书面回答的内容）

StringExample.java 源代码如下：

```
package exp4;

class StringExample
{
    public static void main(String args[])
    {
        String s1 = new String("you are a student"), s2 = new
String("how are you");
        if (s1.equals(s2)) // 使用 equals 方法判断 s1 与 s2 是否相同
        {
```

```
            System.out.println("s1 与 s2 相同");
        } else {
            System.out.println("s1 与 s2 不相同");
        }

        String s3 = new String("22030219851022024");
        if (s3.startsWith("220302")) //判断 s3 的前缀是否是"220302"
        {
            System.out.println("吉林省的身份证");
        }

        String s4 = new String("你"), s5 = new String("我");
        if (s4.compareTo(s5) > 0) //按字典序 s4 大于 s5 的表达式
        {
            System.out.println("按字典序 s4 大于 s5");
        } else {
            System.out.println("按字典序 s4 小于 s5");
        }

        int position = 0;
        String path = "c:\\java\\jsp\\A.java";
        position = path.lastIndexOf("\\"); //获取 path 中最后出现目录分隔
符号的位置
        System.out.println("c:\\java\\jsp\\A.java 中最后出现\\的位置:" +
position);

        String fileName = path.substring(position + 1); //获取 path 中
"A.java"子字符串
        System.out.println("c:\\java\\jsp\\A.java 中含有的文件名:" +
fileName);

        String s6 = new String("100"), s7 = new String("123.678");
        int n1 = Integer.parseInt(s6); //将 s6 转化成 int 型数据
        double n2 = Double.parseDouble(s7); //将 s7 转化成 double 型数据
        double m = n1 + n2;
        System.out.println(m);

        String s8 = String.valueOf(m); //将 m 转化为字符串对象
        position = s8.indexOf(".");
        String temp = s8.substring(position + 1);
```

```
        System.out.println("数字" + m + "有" + temp.length() + "位小数
");

        String s9 = new String("ABCDEF");
        char a[] = s9.toCharArray(); //将 s9 存放到数组 a 中
        for (int i = a.length - 1; i >= 0; i--) {
            System.out.print(" " + a[i]);
        }
    }
}
```

Example.java 源代码如下：

```
class People
{
    protected double weight, height;
    public void speakHello()
    {
        System.out.println("yayawawa");
    }
    public void averageHeight()
    {
        height = 173;
        System.out.println("average height:" + height);
    }
    public void averageWeight()
    {
        weight = 70;
        System.out.println("average weight:" + weight);
    }
}

class ChinaPeople extends People
{
    public void speakHello()
    {
        System.out.println("你好，吃饭了吗");
    }
    public void averageHeight()
    {
        height = 173;
        System.out.println("中国人的平均身高： " + height + "厘米");
```

```
    }
    public void averageWeight()
    {
        weight = 67.34;
        System.out.println("中国人的平均体重：" + weight + "公斤");
    }
    public void chinaGongfu()
    {
        System.out.println("坐如钟，站如松，睡如弓");
    }
}

class AmericanPeople extends People
{
    public void speakHello()
    {
        System.out.println("How do you do");
    }
    public void averageHeight()
    {
        height = 180;
        System.out.println("American average height:" + height + "
cm");
    }
    public void averageWeight()
    {
        weight = 80.5;
        System.out.println("American average weight:" + weight + "
kg");
    }
    public void americanBoxing()
    {
        System.out.println("直拳、勾拳");
    }
}

class BeijingPeople extends ChinaPeople
{
    public void speakHello()
    {
```

```
        System.out.println("您好，这里是北京");
    }
    public void averageHeight()
    {
        height = 172.5;
        System.out.println("北京人的平均身高：" + height + "厘米");
    }
    public void averageWeight()
    {
        weight = 70;
        System.out.println("北京人的平均体重：" + weight + "公斤");
    }
    public void beijingOpera()
    {
        System.out.println("京剧术语：生旦净末丑");
    }
}

public class Example
{
    public static void main(String args[])
    {
        ChinaPeople chinaPeople = new ChinaPeople();
        AmericanPeople americanPeople = new AmericanPeople();
        BeijingPeople beijingPeople = new BeijingPeople();

        chinaPeople.speakHello();
        americanPeople.speakHello();
        beijingPeople.speakHello();

        chinaPeople.averageHeight();
        americanPeople.averageHeight();
        beijingPeople.averageHeight();

        chinaPeople.averageWeight();
        americanPeople.averageWeight();
        beijingPeople.averageWeight();

        chinaPeople.chinaGongfu();
        americanPeople.americanBoxing();
```

```
        beijingPeople.beijingOpera();
        beijingPeople.chinaGongfu();
    }
}
```

运行结果如下：

```
"C:\Program Files\Eclipse Adoptium\jdk-21.0.7.6-hotspot\bin\java.exe"
你好，吃饭了吗
How do you do
您好，这里是北京
中国人的平均身高：173.0厘米
American average height:180.0 cm
北京人的平均身高：172.5厘米
中国人的平均体重：67.34公斤
American average weight:80.5 kg
北京人的平均体重：70.0公斤
坐如钟，站如松，睡如弓
直拳、勾拳
京剧术语：生旦净末丑
坐如钟，站如松，睡如弓

进程已结束，退出代码为 0
```

代码主要是学习类的继承关系,通过一个共同的父类 People,实现了三个方法,speakHello,averageHeight 与averageWeight,然后子类ChinaPeople 和AmericanPeople 分别继承 People 并且重写父类的三个方法,同时添加了属于子类的方法.然后 BeijingPeople 继承了 ChinaPeople 重写了父类方法。

至此实验四结束。

5 Java 实验五 包、接口、类库

5.1 实验内容（要求、算法、步骤和方法）

包的创建和引用。

练习内容：创建接口 Speakable 和 Runner，然后创建两个类 Dog 和 Person 实现该接口。

思考问题：运行上面的程序，思考如下的问题：

（1） 该程序编译后生成几个字节码文件？

（2） 创建一个类 Bird（鸟），给出其声音特征，并在主类中创建一个

Bird 类的实例，输出其特征。

（3） 如何编写抽象类代替程序中的接口，实现程序同样的功能。试比较它们的不同。

上机作业：

（1）创建一个名称为 Vehicle 的接口，在接口中添加两个带有一个参数的方法 start()和 stop()。在两个名称分别为 Bike 和 Bus 的类中实现 Vehicle 接口。创建一个名称为 interfaceDemo 的类，在 interfaceDemo 的 main()方法中创建 Bike 和 Bus 对象，并访问 start()和 stopt()方法。

（2）创建一个名称为 MainPackage 的包，使它包含 ParentClass 和 SubClass。ParentClass 包含变量声明，其值从构造函数中输出。SubClass 类从父类派生而来，完成对父类变量的赋值。创建一个名称为 DemoPackage 的主类，使它不在 MainPackage 包中，在该类中创建一个 SubClass 类的对象。

其它请参见实验指导书。

5.2 Java 源代码文件说明

具体代码请参见附件中电子版的 java 源程序。源程序的简要说明如下：

DefiPackage.java——从实验指导书中得到的,用于创建软件 package 以及一个输出方法.

TestPackage.java——从实验指导书中得到的代码，用于测试 DefiPackage.java 程序

5.3 Java 实验结果

(包括重点代码说明，思路，运行结果，以及实验要求中需要书面回答的内容)

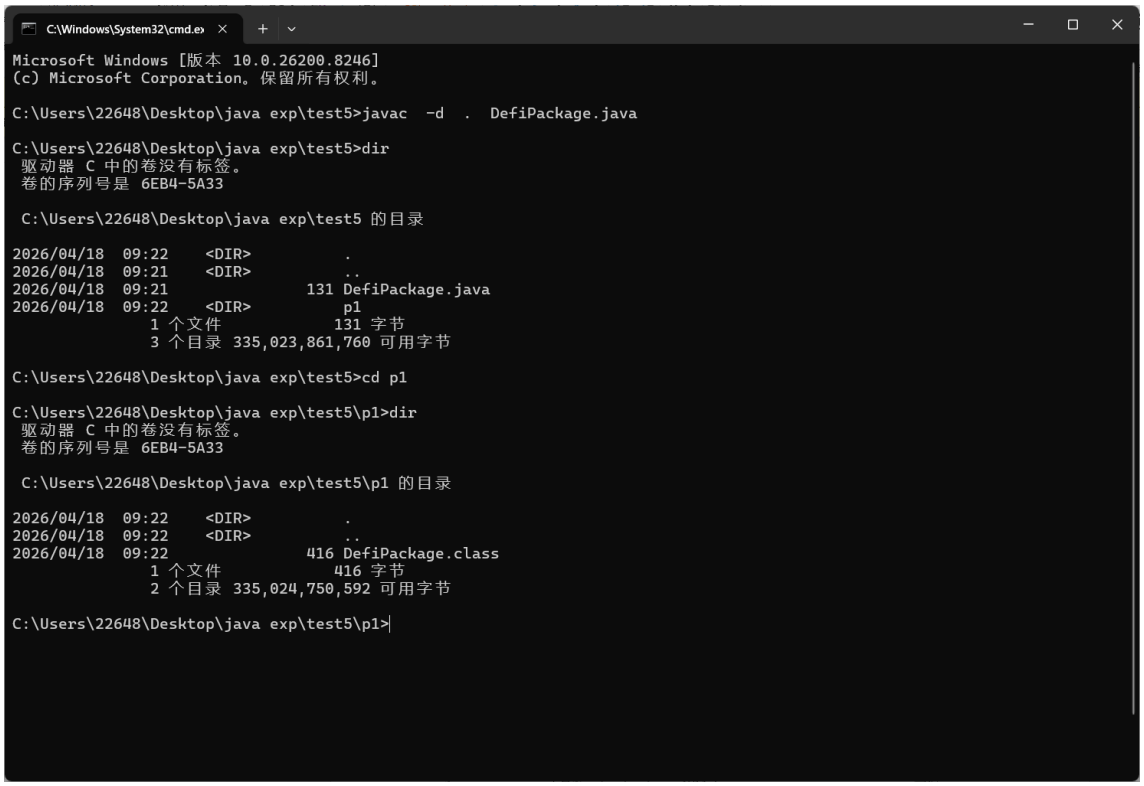
DefiPackage.java 源代码如下：

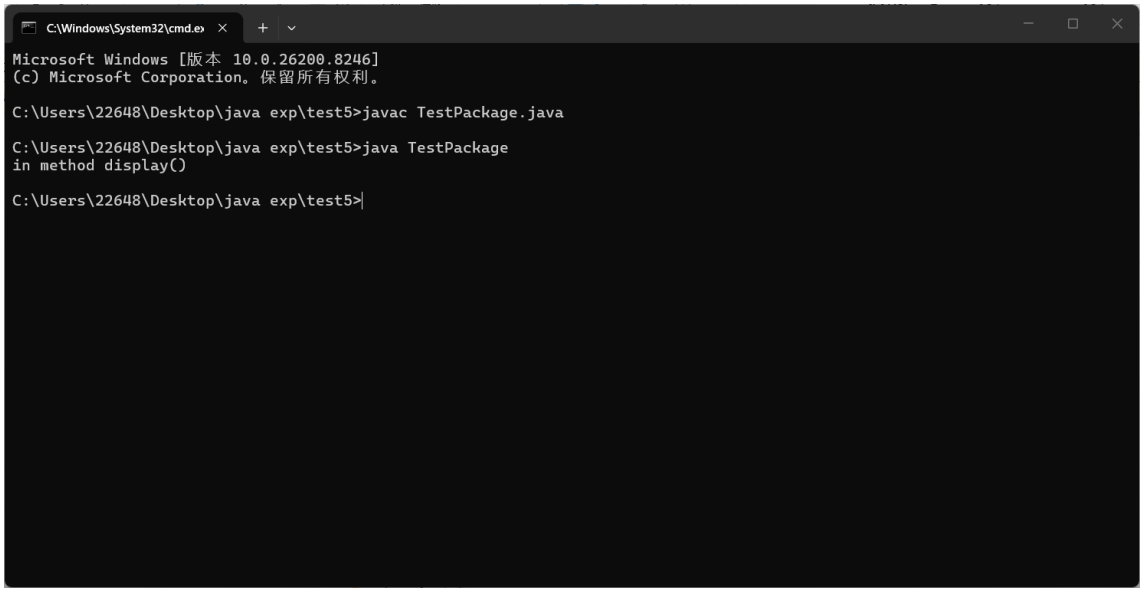
```
package p1;
public class DefiPackage
{
    public void display()
    {
        System.out.println("in method display()");
    }
}
```

```java
package p2;
import p1.DefiPackage;

public class TestPackage
{
    public static void main(String[] args)
    {
        DefiPackage t=new DefiPackage();
        t.display();
    }
}
```

运行结果如下：





可以看到,在编译 DefiPackage. java 过程中产生的. class 文件进入 p1 当中,同时运行 TestPackage. java 得到结果。

思考问题的源码如下:

```
package exp5;

interface Speakable{
    public void speak();
}
interface Runner{
    public void run();
}
class Dog implements Speakable,Runner{
    public void speak(){
        System.out.println("狗的声音:汪、汪! ");
    }
    public void run(){
        System.out.println("狗用四肢跑步");
    }
}
class Person implements Speakable,Runner{
    public void speak(){
        System.out.println("人们见面时经常说:您好! ");
    }
    public void run(){
        System.out.println("人用两腿跑步");
    }
```

```
}
public class TestInterface{
    public static void main(String[] args) {
        Dog d=new Dog();
        d.speak(); d.run();
        Person p=new Person();
        p.speak();  p.run();
    }
}
```

（1） 编译过后生成 4 个字节码文件。

（2） 添加的 Bird 如下：

```
class Bird implements Speakable, Runner
{
    public void speak() {System.out.println("鸟的声音:布谷");}
    public void run() {System.out.println("鸟用翅膀飞");}
}
```

（3） 抽象类代替程序中的接口代码如下：

```
abstract class Speaker {
    public abstract void speak();
}

abstract class Runner {
    public abstract void run();
}

class Dog extends Speaker {
    public void speak() {
        System.out.println("狗的声音:汪、汪！ ");
    }
    private Runner runner = new Runner() {
        public void run() {
            System.out.println("狗用四肢跑步");
        }
    };

    public void run() {
        runner.run();
    }
}
```

```
class Person extends Speaker {
    public void speak() {
        System.out.println("人们见面时经常说:您好！ ");
    }

    private Runner runner = new Runner() {
        public void run() {
            System.out.println("人用两腿跑步");
        }
    };

    public void run() {
        runner.run();
    }
}

class Bird extends Speaker {
    public void speak() {
        System.out.println("鸟的声音:布谷");
    }

    private Runner runner = new Runner() {
        public void run() {
            System.out.println("鸟用翅膀飞");
        }
    };

    public void run() {
        runner.run();
    }
}

public class TestAbstractClass_abstract {
    public static void main(String[] args) {
        Dog d = new Dog();
        d.speak(); d.run();
        Person p = new Person();
        p.speak(); p.run();
        Bird b = new Bird();
```

```
        b.speak(); b.run();
    }
}
```

利用抽象类与接口不同的是，一个类可以实现多个接口，但是 Java 不支持多继承，所以不能同时继承两个抽象类. 只能通过组合的方式来实现原有方式. 接口只能包含抽象方法与常量，而抽象类可以包含具体方法，抽象方法，成员变量和构造方法. 接口更适合是实现多重行为，抽象类更适合共享代码。

上机作业的代码如下：

（1）

```
package exp5;
interface Vehicle
{
    void start(String name);
    void stop(String name);
}

class Bike implements Vehicle
{
    public void start(String name) {
        System.out.println(name + "自行车开始");
    }
    public void stop(String name) {
        System.out.println(name + "自行车停止");
    }
}

class Bus implements Vehicle
{
    public void start(String name) {
        System.out.println(name + "公交车开始");
    }
    public void stop(String name) {
        System.out.println(name + "公交车到站");
    }
}

public class InterfaceTest
```

```
{
    public static void main(String[] args)
    {
        Bike bike = new Bike();
        Bus bus = new Bus();

        bike.start("test");
        bike.stop("test");

        bus.start("207 路");
        bus.stop("207 路");
    }
}
```

（2）

```
package MainPackage;
public class SubClass extends ParentClass
{
    public SubClass(String value)
    {
        super(value); // 调用父类构造函数完成赋值
        System.out.println("子类赋值完成: " + parentVar);
    }
}

package MainPackage;

public class ParentClass
{
    protected String parentVar;
    public ParentClass(String value) {
        this.parentVar = value;
        System.out.println("父类构造函数输出: " + parentVar);
    }
}


import MainPackage.SubClass;

public class DemoPackage {
    public static void main(String[] args) {
```

```
        SubClass sub = new SubClass("这是父类变量的值");
    }
}
```

至此实验五结束。

6 实验总结

(每次实验课写一个实验总结)

通过这次实验,将理论课上学到的 Java 的基本理论重新复习了一下,通过实验将理论应用在实践上对于面向对象(OOP)的理解更加深刻,将其于 C 语言的面向过程的语言进行了对比之后,面向对象的编程范式更加符合人类的直觉,有更好的开发效率.将 Java 的基本操作,面向对象编程范式,类的方法与属性,类的继承,接口,包,和库的知识进行了巩固,为后面利用 Java 开发大型程序打下了基础。

6.1 实验中遇到的问题以及解决方法

(如果在实验过程中遭遇问题，请在这里记录具体问题以及采取的解决方案。)

1. 第一个实验当中,将指导书上的代码复制之后,发现无法通过编译,经过测试之后发现,是指导书上的代码有错误,经过修改之后得以成功运行
2. 第二个实验的练习二中,在将变量 pi 修改为 final 类型之后,编译出错.经过检查发现,在 Ccircle 的 setCircle 方法中对 pi 进行了修改,而 final 变量是无法修改的,经过修改之后,可以正常运行.
3. 第二个实验的上机作业中,发现用命令行运行的 Java 程序无法输出中文,经过上网搜寻发现是编码不同导致的,将命令行终端的编码换成 UTF-8 即可解决.
4. 第五个实验的练习(3)中,直接将接口使用抽象类替代之后,发现无法正常通过编译,在翻阅资料之后,发现 Java 中一个类可以实现多个接口但是不能同时继承多个类,所以无法正常通过编译.于是重新组织代码,利用组合的方式来让一个类可以实现两个抽象类的方法.成功通过编译

6.2 实验体会

（写下实验的收获、了解到的新知识、理解的概念、掌握的技能以及反思。如果有对未来实验或者实践项目的建议，也可以在这里提出。）

通过这五次 Java 实验课程的学习，我逐渐从对编程语言的陌生状态过渡到能够独立完成基础项目的开发。初次接触 Java 时，面对命令行编译、类与对象等概念总感觉有些抽象，特别是在实验一用记事本手写代码并手动编译时，常常因为拼写错误或路径问题导致程序无法运行。但随着实验课程的推进，我开始理解"万物皆对象"的设计理念，发现用面向对象的方式组织代码能让程序结构更清晰。在实现学生 GPA 计算系统时，第一次感受到将现实世界的"学生"、"课程"转化为 Java 类过程。

实验过程中 Java 中比较重要的是类与类之间的关系。在实现"China People"和"AmericanPeople"的继承时，当看到通过方法重写让不同子类对象调用相同方法名却产生不同输出时，领悟到多态的优异。这种将共性抽取到父类、特性保留在子类的设计方式，虽然在使用 ArrayList 集合时曾因泛型类型不匹配报错，但通过查阅 API 文档和调试，逐渐掌握了集合框架的使用技巧。

通过这系列实验，我不仅掌握了 Java 基础语法，更重要的是培养了编程思维。从最初只会照搬实验手册代码，到现在能够根据需求自主设计类结构、选择合适的数据类型.总而言之,这次实验收获是非常大的,我也希望能够有朝一日使用 Java 开发大型项目。